



LAB 3 – Documentation

When working on a project that needs to be completed within a certain time frame, apart from code reviews, automation tests, unit tests, and many more things, rarely do we have time left for documentation. And no matter what you're developing, sooner or later you and your colleagues are going to visit that piece of code again. And when the day arrives, most of us are going to get lost in those code blocks!

Documentation is left out because of the time it consumes, but what if all this can be automated and within a glimpse of an eye you can generate a beautiful website that documents your entire code? This is where Docstring and Sphinx comes in!

Docstring

Python documentation string or commonly known as docstring, is a string literal, and it is used in the class, module, function, or method definition. Docstrings are accessible from the doc attribute (`__doc__`) for any of the Python objects and also with the built-in `help()` function. An object's docstring is defined by including a string constant as the first statement in the object's definition.

Docstrings are great for understanding the functionality of the larger part of the code, i.e., the general purpose of any class, module, or function, whereas the comments are used for code, statements, and expressions, which tend to be small. They are descriptive text written by a programmer mainly for themselves to know what the line of code or expression does and also for the developer who wishes to contribute to that project. It is an essential part that documenting your Python code is going to serve well enough for writing clean code and well-written programs.

Docstrings help you understand the capabilities of a module or a function. For example, let's say you installed the scikit-learn library, and you would like to know all about the sklearn package like description, package modules, etc., you could simply use the help function to get all the information.

```
import sklearn
```

```
help(sklearn)
```

Python Docstrings vs. Commenting

Docstrings are similar in spirit to commenting, but they are enhanced, more logical, and useful versions of commenting. Docstrings act as documentation for the class, module, and packages.

On the other hand, Comments are mainly used to explain non-obvious portions of the code and can be useful for comments on Fixing bugs and tasks that are needed to be done.

Docstrings are represented with closing & opening quotes, while comments start with a # at the beginning.



Note that comments can not be accessed with the built-in doc attribute and help function. Let's see what happens if you try doing so:

```
def string_reverse(str1):  
  
    #Returns the reversed String.  
  
  
    #Parameters:  
  
    # str1 (str):The string which is to be reversed.  
  
  
    #Returns:  
  
    # reverse(str1):The string which gets reversed.  
  
  
    reverse_str1 = ""  
    i = len(str1)  
    while i > 0:  
        reverse_str1 += str1[i - 1]  
        i = i - 1  
    return reverse_str1  
print(string_reverse.__doc__)
```

There are a couple of ways of writing or using a Docstring, i.e., one-line docstring and multi-line docstring. Let's learn them one by one.

One-Line Docstring

The one-line Docstrings are the Docstrings, which fits all in one line. You can use one of the quotes, i.e., triple single or triple-double quotes, and opening quotes and closing quotes need to be the same. In the one-line Docstrings, closing quotes are in the same line as with the opening quotes. Also, the standard convention is to use the triple-double quotes.



```
def square(a):
```

```
    """Returned argument a is squared."""
```

```
    return a**a
```

```
print(square.__doc__)
```

Multi-Line Docstring

Multi-line Docstrings also contains the same string literals line as in One-line Docstrings, but it is followed by a single blank along with the descriptive text.

The general format for writing a Multi-line Docstring is as follows:

```
def some_function(argument1):
```

```
    """Summary or Description of the Function
```

```
    Parameters:
```

```
    argument1 (int): Description of arg1
```

```
    Returns:
```

```
    int:Returning value
```

```
    """
```

```
    return argument1
```

```
print(some_function.__doc__)
```

Python Built-in Docstring

Let's view the built-in Python Docstrings.

All the built-in functions, classes, methods have the actual human description attached to it. You can access it in one of two ways.

- doc attribute
- The help function

You would notice that the output of the help function is more verbose than the `__doc__` attribute.

For example:



```
import math  
  
print(math.__doc__)
```

Python Docstring in Classes

In a class definition, a docstring can be used to provide documentation for the class as a whole. You typically place it immediately after the class definition, and it is enclosed in triple quotes ("""). For example:

```
class MyClass:  
  
    """This is the documentation for MyClass."""  
  
    def __init__(self):  
  
        """This is the documentation for the __init__ method."""  
  
        pass
```

Sphinx Style

Sphinx is the easy and traditional style, verbose, and was initially created specifically for Python Documentation. Sphinx uses a reStructured Text, which is similar in usage to Markdown.

```
class Vehicle(object):  
  
    """  
  
    The Vehicle object contains lots of vehicles  
  
    :param arg: The arg is used for ...  
  
    :type arg: str  
  
    :param `*args`: The variable arguments are used for ...  
  
    :param `**kwargs`: The keyword arguments are used for ...  
  
    :ivar arg: This is where we store arg  
  
    :vartype arg: str  
  
    """
```



```
def __init__(self, arg, *args, **kwargs):
    self.arg = arg

def cars(self, distance, destination):
    """We can't travel a certain distance in vehicles without fuels, so here's the fuels

    :param distance: The amount of distance traveled
    :type amount: int
    :param bool destinationReached: Should the fuels be refilled to cover required distance?
    :raises: :class:`RuntimeError`: Out of fuel

    :returns: A Car mileage
    :rtype: Cars
    """
    pass
```

Shinx style for a function

```
"""[Summary]

:param [ParamName]: [ParamDescription], defaults to [DefaultParamVal]
:type [ParamName]: [ParamType](, optional)
...
:raises [ErrorType]: [ErrorDescription]
...
:return: [ReturnDescription]
:rtype: [Return Type]
"""
```

Shinx style for a class

```
class SimpleBleDevice(object):
    """This is a conceptual class representation of a simple BLE device
    (GATT Server). It is essentially an extended combination of the
    :class:`bluepy.btle.Peripheral` and :class:`bluepy.btle.ScanEntry` classes
```



```
:param client: A handle to the :class:`simpleble.SimpleBleClient` client  
object that detected the device  
:type client: class:`simpleble.SimpleBleClient`  
:param addr: Device MAC address, defaults to None  
:type addr: str, optional  
:param addrType: Device address type - one of ADDR_TYPE_PUBLIC or  
ADDR_TYPE_RANDOM, defaults to ADDR_TYPE_PUBLIC  
:type addrType: str, optional  
:param iface: Bluetooth interface number (0 = /dev/hci0) used for the  
connection, defaults to 0  
:type iface: int, optional  
:param data: A list of tuples (adtype, description, value) containing the  
AD type code, human-readable description and value for all available  
advertising data items, defaults to None  
:type data: List, optional  
:param rssi: Received Signal Strength Indication for the last received  
broadcast from the device. This is an integer value measured in dB,  
where 0 dB is the maximum (theoretical) signal strength, and more  
negative numbers indicate a weaker signal, defaults to 0  
:type rssi: int, optional  
:param connectable: `True` if the device supports connections, and `False`  
otherwise (typically used for advertising 'beacons').,  
defaults to `False`  
:type connectable: bool, optional  
:param updateCount: Integer count of the number of advertising packets  
received from the device so far, defaults to 0  
:type updateCount: int, optional  
"""  
  
def __init__(self, client, addr=None, addrType=None, iface=0,  
            data=None, rssi=0, connectable=False, updateCount=0):  
    """Constructor method  
    """  
  
    super().__init__(deviceAddr=None, addrType=addrType, iface=iface)  
    self.addr = addr  
    self.addrType = addrType  
    self.iface = iface  
    self.rssi = rssi  
    self.connectable = connectable  
    self.updateCount = updateCount  
    self.data = data  
    self._connected = False  
    self._services = []  
    self._characteristics = []  
    self._client = client  
  
def getServices(self, uuids=None):  
    """Returns a list of :class:`bluepy.blte.Service` objects representing  
the services offered by the device. This will perform Bluetooth service  
discovery if this has not already been done; otherwise it will return a  
cached list of services immediately..  
  
:param uuids: A list of string service UUIDs to be discovered,  
defaults to None  
:type uuids: List, optional  
:return: A list of the discovered :class:`bluepy.blte.Service` objects,  
which match the provided ``uuids``  
:rtype: List On Python 3.x, this returns a dictionary view object,
```



not a list

```
"""
self._services = []
if(uuids is not None):
    for uuid in uuids:
        try:
            service = self.getServiceByUUID(uuid)
            self.services.append(service)
        except BTLEException:
            pass
    else:
        self._services = super().getServices()
return self._services

def setNotificationCallback(self, callback):
    """Set the callback function to be executed when the device sends a
    notification to the client.

    :param callback: A function handle of the form
        ``callback(client, characteristic, data)``, where ``client`` is a
        handle to the :class:`simpleble.SimpleBleClient` that invoked the
        callback, ``characteristic`` is the notified
        :class:`bluepy.bt.le.Characteristic` object and data is a
        `bytearray` containing the updated value. Defaults to None
    :type callback: function, optional
    """
    self.withDelegate(
        SimpleBleNotificationDelegate(
            callback,
            client=self._client
        )
    )

def getCharacteristics(self, startHnd=1, endHnd=0xFFFF, uuids=None):
    """Returns a list containing :class:`bluepy.bt.le.Characteristic`
    objects for the peripheral. If no arguments are given, will return all
    characteristics. If startHnd and/or endHnd are given, the list is
    restricted to characteristics whose handles are within the given range.

    :param startHnd: Start index, defaults to 1
    :type startHnd: int, optional
    :param endHnd: End index, defaults to 0xFFFF
    :type endHnd: int, optional
    :param uuids: a list of UUID strings, defaults to None
    :type uuids: list, optional
    :return: List of returned :class:`bluepy.bt.le.Characteristic` objects
    :rtype: list
    """
    self._characteristics = []
    if(uuids is not None):
        for uuid in uuids:
            try:
                characteristic = super().getCharacteristics(
                    startHnd, endHnd, uuid)[0]
                self._characteristics.append(characteristic)
            except BTLEException:
                pass
    else:
        self._characteristics = super().getCharacteristics(startHnd,
```



```
endHnd)
    return self._characteristics

def connect(self):
    """Attempts to initiate a connection with the device.

    :return: `True` if connection was successful, `False` otherwise
    :rtype: bool
    """
    try:
        super().connect(self.addr,
                        addrType=self.addrType,
                        iface=self.iface)
    except BLEException as ex:
        self._connected = False
        return (False, ex)
    self._connected = True
    return True

def disconnect(self):
    """Drops existing connection to device
    """
    super().disconnect()
    self._connected = False

def isConnected(self):
    """Checks to see if device is connected

    :return: `True` if connected, `False` otherwise
    :rtype: bool
    """
    return self._connected

def printInfo(self):
    """Print info about device
    """
    print("Device %s (%s), RSSI=%d dB" %
          (self.addr, self.addrType, self.rssi))
    for (adtype, desc, value) in self.data:
        print("  %s = %s" % (desc, value))
```

Why to use Sphinx

In simplest terms, the sphinx takes in your .rst files and converts them to HTML, and all that is done using just a bunch of commands! Major Python libraries like Django, NumPy, SciPy, Scikit-Learn, Matplotlib, and many more are written using Sphinx. Now, it's our turn to use it, let's dive in.

Installing Sphinx:

```
pip install sphinx sphinx_rtd_theme
```

This will install the sphinx package and along with it the theme(optional) which will make our website shine out.



Create a project, out a testing class and function in it. Add Docstring Shinx style comments in them

In you project directory, you should create a “docs” directory

Run the below command inside your docs folder

sphinx-quickstart

After running the command, accept the defaults.

sphinx-quickstart is an interactive tool that asks some questions about your project and then generates a complete documentation directory along with a make.bat file, which will be used later to generate HTML.

Go to conf.py file and make it similar to the below

Configuration file for the Sphinx documentation builder.

#

This file only contains a selection of the most common options. For a full

list see the documentation:

<https://www.sphinx-doc.org/en/master/usage/configuration.html>

-- Path setup -----

If extensions (or modules to document with autodoc) are in another directory,

add these directories to sys.path here. If the directory is relative to the

documentation root, use os.path.abspath to make it absolute, like shown here.

#

import os

import sys

sys.path.insert(0, os.path.abspath('.'))

-- Project information this part should keep the information you entered in the quikstart wizard-----



```
project = 'PyQt5/Tkinter'

copyright = '2025, Iman Ghalayini' // change to your name
author = 'Iman Ghalayini' // change to your name


# The full version, including alpha/beta/rc tags
release = '1.0.0'


# -- General configuration -----

# Add any Sphinx extension module names here, as strings. They can be
# extensions coming with Sphinx (named 'sphinx.ext.*') or your custom
# ones.
extensions = [
    'sphinx.ext.autodoc',
    'sphinx.ext.viewcode',
    'sphinx.ext.napoleon'
]

# Add any paths that contain templates here, relative to this directory.
templates_path = ['_templates']

# List of patterns, relative to source directory, that match files and
# directories to ignore when looking for source files.
# This pattern also affects html_static_path and html_extra_path.
exclude_patterns = ['_build', 'Thumbs.db', '.DS_Store']
```



-- Options for HTML output -----

The theme to use for HTML and HTML Help pages. See the documentation for

a list of builtin themes.

#

html_theme = 'sphinx_rtd_theme'

Add any paths that contain custom static files (such as style sheets) here,

relative to this directory. They are copied after the builtin static files,

so a file named "default.css" will overwrite the builtin "default.css".

html_static_path = ['_static']

Go to your project's parent folder and run the command

sphinx-apidoc -o docs .

In this command, we tell sphinx to grab our code from the maths folder and output the generated .rst files in the docs folder. After this command, you'll see the your python class and function files with .rst extension and modules.rst generated in your docs.

Step 4: Including module.rst and generating html



Now, include the generated modules.rst file in your

```
.. Math documentation master file, created by
sphinx-quickstart on Fri Mar 18 18:07:49 2022.
You can adapt this file completely to your liking, but it should at least
contain the root `toctree` directive.

Welcome to Math's documentation!
=====

.. toctree::
   :maxdepth: 2
   :caption: Contents:

   modules

Indices and tables
=====

* :ref:`genindex`
* :ref:`modindex`
* :ref:`search`
```

index.rst

And then we're all set to generate the beautiful documentation, go inside of your docs folder and run the command

make html

That's it!!!. You can see the HTML files generated inside of your _build folder in docs and the HTML will look like

Now, suppose you did some changes to your code and now you want to rebuild that HTML, Go to your docs folder and write

make clean html

followed by

make html



**AMERICAN
UNIVERSITY
OF BEIRUT**

Faculty of Engineering and Architecture
Software Tools Lab - Fall 2025-2026

Exercise to Be Graded:

Create the documentation for either PyQt GUI code or Tkinter GUI code in Lab2 project as you choose between you and your colleague in your team work. Do not forget to enter your name, project name, copyright, version, etc. upon configuration/installation.